

Motivation: Why Inexact Line Search?

Exact line search: $\lambda_k = \arg \min_{\lambda \geq 0} f(x_k + \lambda \mathbf{d}_k)$

Problems:

- Expensive for large-scale problems (requires solving 1D optimization subproblem)
- Overkill: we don't need the **exact** minimum, just **sufficient progress**
- For complex functions, 1D minimization itself is iterative

Solution: Accept any λ_k that satisfies **Wolfe conditions**

- Sufficient decrease (Armijo condition): guarantee progress
- Curvature condition: avoid tiny steps

Benefit: Practical algorithms use backtracking line search (much cheaper than exact line search).

Wolfe Conditions

Let $c_1, c_2 \in (0, 1)$ with $c_1 < c_2$. A step size $\lambda > 0$ satisfies the **Wolfe conditions** if:

Armijo (Sufficient Decrease):

$$f(x_k + \lambda \mathbf{d}_k) \leq f(x_k) + c_1 \lambda \nabla f(x_k)^T \mathbf{d}_k$$

Curvature Condition:

$$\nabla f(x_k + \lambda \mathbf{d}_k)^T \mathbf{d}_k \geq c_2 \nabla f(x_k)^T \mathbf{d}_k$$

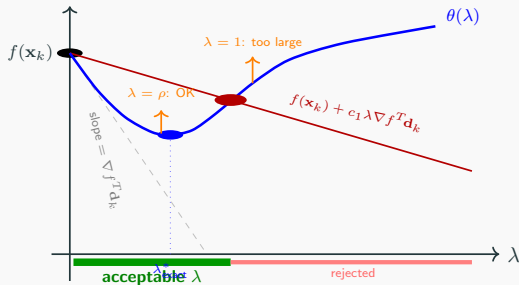
Typical values: $c_1 = 10^{-4}$ and $c_2 = 0.9$ (or 0.1 for Newton methods)

Interpretation:

- **Armijo:** New function value not too much higher than linear approximation
- **Curvature:** Slope at $x_k + \lambda \mathbf{d}_k$ less steep than at x_k (**prevents tiny steps**)

Armijo Condition: Geometric Picture

Accept λ if $\theta(\lambda) := f(\mathbf{x}_k + \lambda \mathbf{d}_k) \leq f(\mathbf{x}_k) + c_1 \lambda \nabla f(\mathbf{x}_k)^T \mathbf{d}_k$.



- **Blue curve:** $\theta(\lambda) = f(\mathbf{x}_k + \lambda \mathbf{d}_k)$ **Red line:** Armijo upper bound
- **Acceptable:** λ where $\theta(\lambda)$ lies below the red line
- **Backtracking:** start at $\lambda = 1$, multiply by $\rho \in (0, 1)$ until Armijo holds

Sufficient Decrease: Textbook Visualization

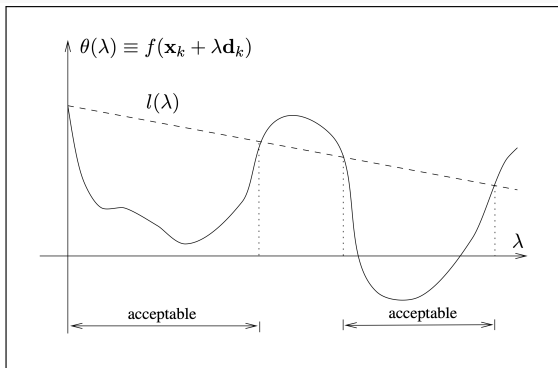


Figure 3.3 Sufficient decrease condition.

The Armijo condition defines an **acceptable region** for the step size. The linear function $f(x_k) + c_1 \lambda \nabla f(x_k)^T \mathbf{d}_k$ is much less steep than the actual function (with $c_1 = 10^{-4}$).

Curvature Condition: Visualization

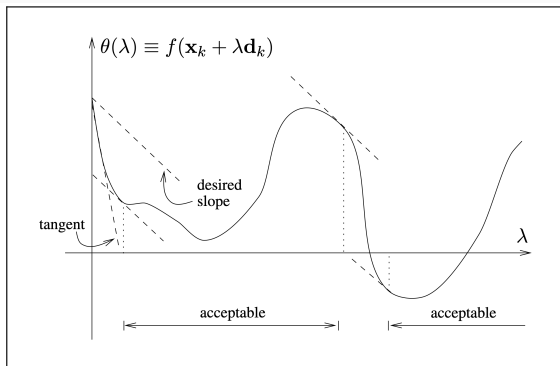


Figure 3.4 The curvature condition.

The curvature condition ensures the directional derivative increases (becomes less negative), preventing us from stopping at tiny step sizes where we could still improve significantly.

Backtracking Line Search Algorithm

Algorithm: Backtracking Line Search (Armijo Condition)

Input: $\mathbf{x}_k$, descent direction $\mathbf{d}_k$, parameters $c_1 \in (0, 0.5)$, $\rho \in (0, 1)$.

Set $\lambda = 1$ (start with full step).

1. If $f(\mathbf{x}_k + \lambda \mathbf{d}_k) \leq f(\mathbf{x}_k) + c_1 \lambda \nabla f(\mathbf{x}_k)^T \mathbf{d}_k$, **return** λ .
2. Set $\lambda \leftarrow \rho \lambda$ (reduce step size). Go to Step 1.

Common parameters:

- $c_1 = 10^{-4}$ (very small, almost any progress counts)
- $\rho = 0.5$ (halve step size each iteration)
- Typically terminates in 1-5 iterations

Note: This implements only Armijo. Strong Wolfe adds the curvature condition, which requires a different algorithm (e.g., zoom procedure).

Exercise 2: Backtracking Line Search

Backtracking Line Search

Let $f(x_1, x_2) = e^{x_1+x_2} + e^{x_1-x_2} + x_1^2$ and $x_k = (0, 0)^T$. The descent direction is $\mathbf{d}_k = -\nabla f(x_k) = (-2, 0)^T$ (verify this).

Starting with $\lambda = 1$, $c_1 = 10^{-4}$, $\rho = 0.5$:

1. Compute $\nabla f(x_k)^T \mathbf{d}_k$
2. Check if Armijo condition is satisfied for $\lambda = 1$
3. If not, compute the backtracking iterations until it is satisfied

Solution: Backtracking Line Search

Answers

$$f(x_1, x_2) = e^{x_1+x_2} + e^{x_1-x_2} + x_1^2, x_k = (0, 0)^T$$

$$\nabla f = (e^{x_1+x_2} + e^{x_1-x_2} + 2x_1, e^{x_1+x_2} - e^{x_1-x_2})^T$$

$$\text{At } x_k = (0, 0): \nabla f(x_k) = (2, 0)^T, \text{ so } \mathbf{d}_k = (-2, 0)^T$$

$$(1) \nabla f(x_k)^T \mathbf{d}_k = (2, 0)^T \cdot (-2, 0)^T = -4 \checkmark$$

(2) Armijo RHS at $\lambda = 1$:

$$f(x_k) + c_1 \lambda \nabla f(x_k)^T \mathbf{d}_k = 2 + 10^{-4} \cdot 1 \cdot (-4) = 1.9996$$

$$\text{At } x_k + 1 \cdot \mathbf{d}_k = (0, 0)^T + (-2, 0)^T = (-2, 0)^T:$$

$$f(-2, 0) = e^{-2} + e^{-2} + 4 = 2e^{-2} + 4 \approx 4.27 > 1.9996 \quad \text{Armijo}$$

FAILS!

(3) Backtrack: set $\lambda = \rho \cdot 1 = 0.5$. Armijo RHS:

$$2 + 10^{-4}(0.5)(-4) = 1.9998.$$

$$\text{At } x_k + 0.5 \mathbf{d}_k = (-1, 0)^T: f(-1, 0) = 2e^{-1} + 1 \approx 1.736 < 1.9998 \checkmark$$

Armijo satisfied at $\lambda = 0.5$.

Wolfe Conditions: A Deeper Look

Recall the Wolfe conditions for step size $\lambda > 0$ with $0 < c_1 < c_2 < 1$:

1. Armijo (Sufficient Decrease):

$$f(\mathbf{x}_k + \lambda \mathbf{d}_k) \leq f(\mathbf{x}_k) + c_1 \lambda \nabla f(\mathbf{x}_k)^T \mathbf{d}_k$$

2. Curvature Condition:

$$\nabla f(\mathbf{x}_k + \lambda \mathbf{d}_k)^T \mathbf{d}_k \geq c_2 \nabla f(\mathbf{x}_k)^T \mathbf{d}_k$$

Why do we need **both** conditions?

- **Armijo alone** is always satisfied by sufficiently small λ (Taylor's theorem). So it doesn't prevent steps that are **too short**.
- **Simple decrease** ($f(\mathbf{x}_{k+1}) < f(\mathbf{x}_k)$) is not enough for convergence — decrease can shrink so fast the iterates converge to a non-stationary point.
- The **curvature condition** forces you to go far enough that the function isn't still dropping steeply.

Curvature Condition: Intuition

Define $\phi(\lambda) = f(\mathbf{x}_k + \lambda \mathbf{d}_k)$. Then:

- $\phi'(\lambda) = \nabla f(\mathbf{x}_k + \lambda \mathbf{d}_k)^T \mathbf{d}_k$
- $\phi'(0) = \nabla f(\mathbf{x}_k)^T \mathbf{d}_k < 0$ (since $\mathbf{d}_k$ is a descent direction)

The curvature condition says:

$$\phi'(\lambda_k) \geq c_2 \phi'(0)$$

Since $\phi'(0)$ is **negative** and $c_2 < 1$, the RHS $c_2 \phi'(0)$ is **less negative**.

Translation: At the accepted step, the slope must have become **less steep**. If the function is still plummeting along $\mathbf{d}_k$, you should keep going — you're leaving progress on the table.

Why $c_1 < c_2$? If $c_1 \geq c_2$, no step size might satisfy both conditions simultaneously. The gap between c_1 and c_2 creates a region where both conditions can hold.

Strong Wolfe Conditions

Strong Wolfe Conditions

Replace the curvature condition with:

$$|\nabla f(\mathbf{x}_k + \lambda \mathbf{d}_k)^T \mathbf{d}_k| \leq c_2 |\nabla f(\mathbf{x}_k)^T \mathbf{d}_k|$$

Difference from standard Wolfe:

- Standard curvature allows $\phi'(\lambda_k)$ to be very **positive** (you've overshot past a minimum and the function is climbing steeply)
- Strong Wolfe requires $|\phi'(\lambda_k)|$ to be **small** — you've landed near a local minimizer of ϕ

When does it matter?

- **Newton/BFGS**: Standard Wolfe ($c_2 = 0.9$) usually suffices
- **Conjugate gradient**: **Needs** strong Wolfe with tight $c_2 = 0.1$
 - A large positive slope at the accepted point can **destroy** **conjugacy** of subsequent directions

Existence of Wolfe Step Sizes

Question: Is it guaranteed that a step size satisfying both conditions exists?

Lemma (Nocedal & Wright, Lemma 3.1)

If f is bounded below along the ray $\{\mathbf{x}_k + \lambda \mathbf{d}_k : \lambda \geq 0\}$ and ∇f is continuous, then there exist step sizes satisfying the Wolfe conditions.

Proof sketch:

Step 1: The Armijo line $f(\mathbf{x}_k) + c_1 \lambda \phi'(0)$ decreases linearly. Since f is bounded below, $\phi(\lambda)$ must cross **above** this line at some point. Let λ' be the smallest such crossing.

Step 2: On $[0, \lambda']$, Armijo holds. By the **mean value theorem**, there exists $\lambda^* \in (0, \lambda')$ where:

$$\phi'(\lambda^*) = \frac{\phi(\lambda') - \phi(0)}{\lambda'}$$

Step 3: Since $\phi(\lambda')$ is on the Armijo line, the average slope equals $c_1 \phi'(0)$. Since $c_1 < c_2$, this slope satisfies the curvature condition.

Both Wolfe conditions can always be satisfied simultaneously.

Convergence of Line Search Methods: The Big Question

We've seen that the Wolfe conditions give us a practical line search. But:

Does any descent method with Wolfe line search actually converge?

The Zoutendijk theorem provides the definitive answer. It applies to [any](#) descent method — steepest descent, Newton, BFGS, conjugate gradient — as long as the Wolfe conditions are satisfied.

Assumptions:

1. f is bounded below on $\mathcal{L}_0 = \{\mathbf{x} \in \mathbb{R}^n : f(\mathbf{x}) \leq f(\mathbf{x}_0)\}$
2. ∇f is [Lipschitz continuous](#): there exists $L > 0$ such that

$$\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\| \quad \text{for all } \mathbf{x}, \mathbf{y} \text{ near } \mathcal{L}_0$$

These are mild: any C^2 function on a bounded region satisfies both.

The Zoutendijk Theorem

Define the angle θ_k between $\mathbf{d}_k$ and $-\nabla f(\mathbf{x}_k)$:

$$\cos \theta_k = \frac{-\nabla f(\mathbf{x}_k)^T \mathbf{d}_k}{\|\nabla f(\mathbf{x}_k)\| \|\mathbf{d}_k\|}$$

Theorem (Zoutendijk, 1970)

Under the assumptions above, if each λ_k satisfies the Wolfe conditions, then:

$$\sum_{k=0}^{\infty} \cos^2 \theta_k \|\nabla f(\mathbf{x}_k)\|^2 < \infty$$

Immediate consequence: Since terms of a convergent series tend to zero:

$$\cos^2 \theta_k \|\nabla f(\mathbf{x}_k)\|^2 \rightarrow 0$$

This means [either](#):

- $\cos \theta_k \rightarrow 0$ (search directions become orthogonal to the gradient),
OR
- $\|\nabla f(\mathbf{x}_k)\| \rightarrow 0$ ([convergence to a stationary point](#))

Proof of the Zoutendijk Theorem

Step 1: Armijo gives decrease bound. From Armijo:

$$f(\mathbf{x}_k) - f(\mathbf{x}_{k+1}) \geq c_1 \lambda_k |\nabla f(\mathbf{x}_k)^T \mathbf{d}_k|$$

Step 2: Curvature + Lipschitz give lower bound on λ_k .

The curvature condition yields:

$$[\nabla f(\mathbf{x}_k + \lambda_k \mathbf{d}_k) - \nabla f(\mathbf{x}_k)]^T \mathbf{d}_k \geq (c_2 - 1) \nabla f(\mathbf{x}_k)^T \mathbf{d}_k$$

By Lipschitz continuity, the LHS $\leq L \lambda_k \|\mathbf{d}_k\|^2$. Therefore:

$$\lambda_k \geq \frac{(1 - c_2) |\nabla f(\mathbf{x}_k)^T \mathbf{d}_k|}{L \|\mathbf{d}_k\|^2}$$

Step 3: Combine and telescope. Substituting:

$$f(\mathbf{x}_k) - f(\mathbf{x}_{k+1}) \geq \frac{c_1(1 - c_2)}{L} \cos^2 \theta_k \|\nabla f(\mathbf{x}_k)\|^2$$

Summing from $k = 0$ to N :

$$f(\mathbf{x}_0) - f(\mathbf{x}_{N+1}) \geq \frac{c_1(1 - c_2)}{L} \sum_{k=0}^N \cos^2 \theta_k \|\nabla f(\mathbf{x}_k)\|^2$$

Since f is bounded below, the LHS is bounded. So the series converges.

□

The Angle Condition: Key to Convergence

The Zoutendijk theorem tells us: to prove $\|\nabla f(\mathbf{x}_k)\| \rightarrow 0$, we just need to show

$$\cos \theta_k \geq \delta > 0 \quad \text{for all } k$$

for some fixed δ . This is the **angle condition**.

This is the modular decomposition:

1. **Line search module** (done once, universally): Wolfe conditions $\implies$ Zoutendijk summability condition
2. **Direction module** (per algorithm): Show that $\cos \theta_k$ stays bounded away from zero

Applications to Specific Methods

- **Steepest descent:** $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$, so $\theta_k = 0$, $\cos \theta_k = 1$ always. ✓
- **Newton:** If Hessian eigenvalues in $[m, M]$ with $m > 0$, then $\cos \theta_k \geq m/M$. ✓
- **BFGS:** Showing $\cos \theta_k \geq \delta > 0$ is delicate; requires analysis of the quasi-Newton updates.
- **Conjugate gradient:** Hardest case. May need periodic restarts.

Why the Zoutendijk Theorem Matters

1. Universal convergence guarantee.

Any method producing descent directions with Wolfe line search inherits the summability condition. The theorem is a “meta-theorem” from which convergence of individual methods follows as corollaries.

2. Clean division of labor.

It separates the line search analysis (done once) from the direction analysis (specific to each method). This is how optimization software is designed: the line search is a **self-contained module** paired with any direction-generating scheme.

3. Diagnostic tool.

If an algorithm fails to converge, the Zoutendijk condition tells you **exactly** where the failure lies: $\cos \theta_k \rightarrow 0$, meaning the search directions are becoming orthogonal to the gradient.

4. Connection to BFGS.

The Wolfe curvature condition guarantees $s_k^T y_k > 0$ (where $s_k = \lambda_k \mathbf{d}_k$, $y_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$), which is **required** for the BFGS update to maintain positive definiteness.

Motivation: Newton's Method

Core idea: Minimize the quadratic Taylor approximation

At current point x_k :

$$m_k(d) = f(x_k) + \nabla f(x_k)^T d + \frac{1}{2} d^T \nabla^2 f(x_k) d$$

This quadratic is minimized when:

$$\nabla m_k(d) = \nabla f(x_k) + \nabla^2 f(x_k) d = 0$$

Solving for d :

$$d_k^N = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

Newton step:

$$x_{k+1} = x_k + d_k^N = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

Advantage over steepest descent:

- Uses curvature information (Hessian) $\nabla^2 f$
- Quadratic convergence near the solution
- Converges in one step for convex quadratics!

Newton's Method Algorithm

Algorithm: Newton's Method

Initialization: Let $\epsilon > 0$. Choose a starting point $\mathbf{x}_1 \in \mathbb{R}^n$, set $k = 1$.

1. If $\|\nabla f(\mathbf{x}_k)\|_2 < \epsilon$, **stop**. Otherwise, compute the Newton direction:

$$\mathbf{d}_k = -[\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k).$$

2. (Optional) Line search: $\lambda_k = \text{backtracking}(\mathbf{x}_k, \mathbf{d}_k)$. Otherwise set $\lambda_k = 1$.
3. Set $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{d}_k$. Set $k \leftarrow k + 1$ and go to Step 1.

Computational cost:

- Compute Hessian: $O(n^2)$ or $O(n^3)$ (depends on sparsity)
- Solve linear system: $O(n^3)$ (standard: LU factorization)
- Per iteration: **much more expensive** than steepest descent

Trade-off: Fewer iterations (quadratic convergence) vs. higher per-iteration cost

Proof: Newton Direction is a Descent Direction

Theorem

If $\nabla^2 f(\mathbf{x}_k) \succ 0$ (positive definite) and $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$, then the Newton direction $\mathbf{d}_k^N = -[\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)$ is a descent direction.

Step 1: Compute the directional derivative:

$$\nabla f(\mathbf{x}_k)^T \mathbf{d}_k^N = \nabla f(\mathbf{x}_k)^T (-[\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k))$$

Step 2: Simplify: Let $\mathbf{g} := \nabla f(\mathbf{x}_k) \neq \mathbf{0}$ and $H := \nabla^2 f(\mathbf{x}_k) \succ 0$:

$$\nabla f(\mathbf{x}_k)^T \mathbf{d}_k^N = -\mathbf{g}^T H^{-1} \mathbf{g}$$

Step 3: Use positive definiteness: Since $H \succ 0$, we have $H^{-1} \succ 0$.

For any nonzero $\mathbf{g}$: $\mathbf{g}^T H^{-1} \mathbf{g} > 0$

$$\therefore \nabla f(\mathbf{x}_k)^T \mathbf{d}_k^N = -\mathbf{g}^T H^{-1} \mathbf{g} < 0$$

The Newton direction is a strict descent direction.



Newton's Method: Quadratic Case

For the convex quadratic $f(x) = \frac{1}{2}x^T Qx + c^T x + d$ with $Q \succ 0$:

Step 1: Compute gradient and Hessian:

$$\nabla f(x) = Qx + c, \quad \nabla^2 f(x) = Q \quad (\text{constant!})$$

Step 2: Newton direction at x_0 :

$$d_0^N = -Q^{-1}(Qx_0 + c) = -x_0 - Q^{-1}c$$

Step 3: Newton update with unit step ($\lambda = 1$):

$$x_1 = x_0 + d_0^N = x_0 - x_0 - Q^{-1}c = -Q^{-1}c = x^*$$

This is independent of x_0 !

Newton's method reaches the global optimum in exactly 1 step for any quadratic.

Comparison to steepest descent:

- SD: $O(\kappa(Q) \log(1/\epsilon))$ iterations (slow for ill-conditioned)
- Newton: 1 iteration (for quadratics) (unbeatable!)

Newton's Method: Convergence Rate

Theorem: Newton's Method Convergence

Let f be twice continuously differentiable and x^* be a local minimum with $\nabla^2 f(x^*) \succ 0$. Then there exists a neighborhood of x^* such that, if x_0 is in this neighborhood, Newton's method (without line search) converges to x^* with **quadratic convergence**:

$$\|x_{k+1} - x^*\| \leq C \|x_k - x^*\|^2$$

for some constant $C > 0$.

Interpretation:

- The error **squares** at each iteration (when close to solution)
- Example: $\|x_0 - x^*\| = 10^{-1} \Rightarrow \|x_1 - x^*\| \leq C \cdot 10^{-2} \Rightarrow \|x_2 - x^*\| \leq C \cdot 10^{-4} \Rightarrow \dots$
- **Superfast** once close enough (quadratic convergence)
- **Slow** or **diverges** far from solution

In practice: Use line search to ensure global convergence (convergence from any starting point).

Proof: Quadratic Convergence of Newton (Part 1)

Assumptions:

- (a) $\nabla f(\mathbf{x}^*) = \mathbf{0}$ and $H^* := \nabla^2 f(\mathbf{x}^*) \succ \mathbf{0}$
- (b) $\nabla^2 f$ is **Lipschitz continuous**: $\|\nabla^2 f(\mathbf{x}) - \nabla^2 f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$ near $\mathbf{x}^*$

Proof: Quadratic Convergence of Newton (Part 1)

Assumptions:

- (a) $\nabla f(\mathbf{x}^*) = \mathbf{0}$ and $H^* := \nabla^2 f(\mathbf{x}^*) \succ 0$
- (b) $\nabla^2 f$ is **Lipschitz continuous**: $\|\nabla^2 f(\mathbf{x}) - \nabla^2 f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$ near $\mathbf{x}^*$

Step 1: Error recursion: Let $\mathbf{e}_k := \mathbf{x}_k - \mathbf{x}^*$. The Newton update gives:

$$\begin{aligned}\mathbf{e}_{k+1} &= \mathbf{x}_{k+1} - \mathbf{x}^* = (\mathbf{x}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)) - \mathbf{x}^* \\ &= \mathbf{e}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)\end{aligned}$$

Proof: Quadratic Convergence of Newton (Part 1)

Assumptions:

- (a) $\nabla f(\mathbf{x}^*) = \mathbf{0}$ and $H^* := \nabla^2 f(\mathbf{x}^*) \succ 0$
- (b) $\nabla^2 f$ is **Lipschitz continuous**: $\|\nabla^2 f(\mathbf{x}) - \nabla^2 f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$ near $\mathbf{x}^*$

Step 1: Error recursion: Let $\mathbf{e}_k := \mathbf{x}_k - \mathbf{x}^*$. The Newton update gives:

$$\begin{aligned}\mathbf{e}_{k+1} &= \mathbf{x}_{k+1} - \mathbf{x}^* = (\mathbf{x}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)) - \mathbf{x}^* \\ &= \mathbf{e}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)\end{aligned}$$

Step 2: Integral form of Taylor's theorem: Since $\nabla f(\mathbf{x}^*) = \mathbf{0}$:

$$\nabla f(\mathbf{x}_k) = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}^*) = \underbrace{\left(\int_0^1 \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k) dt \right)}_{=:\tilde{H}_k} \mathbf{e}_k$$

Proof: Quadratic Convergence of Newton (Part 1)

Assumptions:

- (a) $\nabla f(\mathbf{x}^*) = \mathbf{0}$ and $H^* := \nabla^2 f(\mathbf{x}^*) \succ \mathbf{0}$
- (b) $\nabla^2 f$ is **Lipschitz continuous**: $\|\nabla^2 f(\mathbf{x}) - \nabla^2 f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$ near $\mathbf{x}^*$

Step 1: Error recursion: Let $\mathbf{e}_k := \mathbf{x}_k - \mathbf{x}^*$. The Newton update gives:

$$\begin{aligned}\mathbf{e}_{k+1} &= \mathbf{x}_{k+1} - \mathbf{x}^* = (\mathbf{x}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)) - \mathbf{x}^* \\ &= \mathbf{e}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)\end{aligned}$$

Step 2: Integral form of Taylor's theorem: Since $\nabla f(\mathbf{x}^*) = \mathbf{0}$:

$$\nabla f(\mathbf{x}_k) = \nabla f(\mathbf{x}_k) - \nabla f(\mathbf{x}^*) = \underbrace{\left(\int_0^1 \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k) dt \right)}_{=:\overline{H}_k} \mathbf{e}_k$$

Step 3: Substitute into the error recursion:

$$\begin{aligned}\mathbf{e}_{k+1} &= \mathbf{e}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \overline{H}_k \mathbf{e}_k \\ &= [\nabla^2 f(\mathbf{x}_k)]^{-1} [\nabla^2 f(\mathbf{x}_k) - \overline{H}_k] \mathbf{e}_k\end{aligned}$$

Proof: Quadratic Convergence of Newton (Part 2)

Recall: $\mathbf{e}_{k+1} = [\nabla^2 f(\mathbf{x}_k)]^{-1} [\nabla^2 f(\mathbf{x}_k) - \overline{H}_k] \mathbf{e}_k$ where
 $\overline{H}_k = \int_0^1 \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k) dt.$

Proof: Quadratic Convergence of Newton (Part 2)

Recall: $\mathbf{e}_{k+1} = [\nabla^2 f(\mathbf{x}_k)]^{-1} [\nabla^2 f(\mathbf{x}_k) - \overline{H}_k] \mathbf{e}_k$ where

$$\overline{H}_k = \int_0^1 \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k) dt.$$

Step 4: Bound the bracket using Lipschitz continuity:

Since $\mathbf{x}_k = \mathbf{x}^* + \mathbf{e}_k$:

$$\begin{aligned} \|\nabla^2 f(\mathbf{x}_k) - \overline{H}_k\| &= \left\| \int_0^1 [\nabla^2 f(\mathbf{x}^* + \mathbf{e}_k) - \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k)] dt \right\| \\ &\leq \int_0^1 L \|(1-t) \mathbf{e}_k\| dt = L \|\mathbf{e}_k\| \int_0^1 (1-t) dt = \frac{L}{2} \|\mathbf{e}_k\| \end{aligned}$$

Proof: Quadratic Convergence of Newton (Part 2)

Recall: $\mathbf{e}_{k+1} = [\nabla^2 f(\mathbf{x}_k)]^{-1} [\nabla^2 f(\mathbf{x}_k) - \overline{H}_k] \mathbf{e}_k$ where
 $\overline{H}_k = \int_0^1 \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k) dt$.

Step 4: Bound the bracket using Lipschitz continuity:

Since $\mathbf{x}_k = \mathbf{x}^* + \mathbf{e}_k$:

$$\begin{aligned}\|\nabla^2 f(\mathbf{x}_k) - \overline{H}_k\| &= \left\| \int_0^1 [\nabla^2 f(\mathbf{x}^* + \mathbf{e}_k) - \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k)] dt \right\| \\ &\leq \int_0^1 L \|(1-t) \mathbf{e}_k\| dt = L \|\mathbf{e}_k\| \int_0^1 (1-t) dt = \frac{L}{2} \|\mathbf{e}_k\|\end{aligned}$$

Step 5: Take norms:

$$\|\mathbf{e}_{k+1}\| \leq \|[\nabla^2 f(\mathbf{x}_k)]^{-1}\| \cdot \frac{L}{2} \|\mathbf{e}_k\| \cdot \|\mathbf{e}_k\|$$

Proof: Quadratic Convergence of Newton (Part 2)

Recall: $\mathbf{e}_{k+1} = [\nabla^2 f(\mathbf{x}_k)]^{-1} [\nabla^2 f(\mathbf{x}_k) - \overline{H}_k] \mathbf{e}_k$ where
 $\overline{H}_k = \int_0^1 \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k) dt$.

Step 4: Bound the bracket using Lipschitz continuity:

Since $\mathbf{x}_k = \mathbf{x}^* + \mathbf{e}_k$:

$$\begin{aligned}\|\nabla^2 f(\mathbf{x}_k) - \overline{H}_k\| &= \left\| \int_0^1 [\nabla^2 f(\mathbf{x}^* + \mathbf{e}_k) - \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k)] dt \right\| \\ &\leq \int_0^1 L \|(1-t) \mathbf{e}_k\| dt = L \|\mathbf{e}_k\| \int_0^1 (1-t) dt = \frac{L}{2} \|\mathbf{e}_k\|\end{aligned}$$

Step 5: Take norms:

$$\|\mathbf{e}_{k+1}\| \leq \|[\nabla^2 f(\mathbf{x}_k)]^{-1}\| \cdot \frac{L}{2} \|\mathbf{e}_k\| \cdot \|\mathbf{e}_k\|$$

Near $\mathbf{x}^*$: $\|[\nabla^2 f(\mathbf{x}_k)]^{-1}\| \rightarrow \|[H^*]^{-1}\|$, so for $\mathbf{x}_k$ sufficiently close to $\mathbf{x}^*$:

$$\boxed{\|\mathbf{e}_{k+1}\| \leq \underbrace{\frac{L}{2} \|[H^*]^{-1}\|}_{=: C} \|\mathbf{e}_k\|^2}$$

Proof: Quadratic Convergence of Newton (Part 2)

Recall: $\mathbf{e}_{k+1} = [\nabla^2 f(\mathbf{x}_k)]^{-1} [\nabla^2 f(\mathbf{x}_k) - \overline{H}_k] \mathbf{e}_k$ where
 $\overline{H}_k = \int_0^1 \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k) dt$.

Step 4: Bound the bracket using Lipschitz continuity:

Since $\mathbf{x}_k = \mathbf{x}^* + \mathbf{e}_k$:

$$\begin{aligned} \|\nabla^2 f(\mathbf{x}_k) - \overline{H}_k\| &= \left\| \int_0^1 [\nabla^2 f(\mathbf{x}^* + \mathbf{e}_k) - \nabla^2 f(\mathbf{x}^* + t \mathbf{e}_k)] dt \right\| \\ &\leq \int_0^1 L \|(1-t) \mathbf{e}_k\| dt = L \|\mathbf{e}_k\| \int_0^1 (1-t) dt = \frac{L}{2} \|\mathbf{e}_k\| \end{aligned}$$

Step 5: Take norms:

$$\|\mathbf{e}_{k+1}\| \leq \|[\nabla^2 f(\mathbf{x}_k)]^{-1}\| \cdot \frac{L}{2} \|\mathbf{e}_k\| \cdot \|\mathbf{e}_k\|$$

Near $\mathbf{x}^*$: $\|[\nabla^2 f(\mathbf{x}_k)]^{-1}\| \rightarrow \|[\mathbf{H}^*]^{-1}\|$, so for $\mathbf{x}_k$ sufficiently close to $\mathbf{x}^*$:

$$\boxed{\|\mathbf{e}_{k+1}\| \leq \underbrace{\frac{L}{2} \|[\mathbf{H}^*]^{-1}\|}_{=: C} \|\mathbf{e}_k\|^2}$$

Quadratic convergence with constant $C = \frac{L}{2} \|\nabla^2 f(\mathbf{x}^*)^{-1}\|$, provided
 $C \|\mathbf{e}_0\| < 1$. $\square$

Key point: The proof requires $\mathbf{x}_0$ sufficiently close to $\mathbf{x}^*$ — this is **local convergence only**. For global convergence, combine Newton with line search (Modification 1).

Numerical Example: Newton vs. Steepest Descent

Consider:

$$f(x_1, x_2) = (x_1 - 2)^4 + (x_1 - 2x_2)^2$$

Optimal point: $x^* = (2, 1)^T$

Convergence comparison table:

Table 8.12 Summary of Computations for the Method of Newton

Iteration k	x_k $f(x_k)$	$\nabla f(x_k)$	$H(x_k)$	$H(x_k)^{-1}$	$-H(x_k)^{-1}\nabla f(x_k)$	x_{k+1}
1	(0.00, 3.00) 52.00	(-44.0, 24.0)	$\begin{bmatrix} 50.0 & -4.0 \\ -4.0 & 8.0 \end{bmatrix}$	$\frac{1}{384} \begin{bmatrix} 8.0 & 4.0 \\ 4.0 & 50.0 \end{bmatrix}$	(0.67, -2.67)	(0.67, 0.33)
2	(0.67, 0.33) 3.13	(-9.39, -0.04)	$\begin{bmatrix} 23.23 & -4.0 \\ -4.0 & 8.0 \end{bmatrix}$	$\frac{1}{169.84} \begin{bmatrix} 8.0 & 4.0 \\ 4.0 & 23.23 \end{bmatrix}$	(0.44, 0.23)	(1.11, 0.56)
3	(1.11, 0.56) 0.63	(-2.84, -0.04)	$\begin{bmatrix} 11.50 & -4.0 \\ -4.0 & 8.0 \end{bmatrix}$	$\frac{1}{76} \begin{bmatrix} 8.0 & 4.0 \\ 4.0 & 11.50 \end{bmatrix}$	(0.30, 0.14)	(1.41, 0.70)
4	(1.41, 0.70) 0.12	(-0.80, -0.04)	$\begin{bmatrix} 6.18 & -4.0 \\ -4.0 & 8.0 \end{bmatrix}$	$\frac{1}{33.44} \begin{bmatrix} 8.0 & 4.0 \\ 4.0 & 6.18 \end{bmatrix}$	(0.20, 0.10)	(1.61, 0.80)
5	(1.61, 0.80) 0.02	(-0.22, -0.04)	$\begin{bmatrix} 3.83 & -4.0 \\ -4.0 & 8.0 \end{bmatrix}$	$\frac{1}{14.64} \begin{bmatrix} 8.0 & 4.0 \\ 4.0 & 3.83 \end{bmatrix}$	(0.13, 0.07)	(1.74, 0.87)
6	(1.74, 0.87) 0.005	(-0.07, 0.00)	$\begin{bmatrix} 2.81 & -4.0 \\ -4.0 & 8.0 \end{bmatrix}$	$\frac{1}{6.48} \begin{bmatrix} 8.0 & 4.0 \\ 4.0 & 2.81 \end{bmatrix}$	(0.09, 0.04)	(1.83, 0.91)
7	(1.83, 0.91) 0.0009	(0.0003, -0.04)				

Newton's method: Converges in 4-5 iterations with **quadratic convergence**

Steepest descent: Converges in 100+ iterations with **linear convergence**

Newton's Method: Advantages vs. Disadvantages

Advantages:

- Quadratic convergence near solution (extremely fast)
- One step for quadratic problems
- Scale-invariant (unlike steepest descent)
- Naturally handles ill-conditioned problems

Disadvantages:

- High computational cost: $O(n^3)$ per iteration (linear system solve)
- Requires Hessian: May be expensive to compute or unavailable
- May not be PD: If Hessian not positive definite, method can fail
- Local convergence: Needs good starting point (or line search)
- Storage: $O(n^2)$ space for Hessian matrix

When to use:

- Small to medium problems ($n \lesssim 10^4$)
- Hessian available or cheap to compute
- Willing to use line search for robustness

Exercise 3: One Step of Newton's Method

One Step of Newton's Method

For $f(x_1, x_2) = x_1^2 + 4x_2^2$ with $x_0 = (4, 2)^T$:

1. Compute $\nabla f(x_0)$ and $\nabla^2 f(x_0)$
2. Compute the Newton direction $d_0^N = -[\nabla^2 f(x_0)]^{-1} \nabla f(x_0)$
3. Compute $x_1 = x_0 + d_0^N$
4. Compare with steepest descent: which reaches the optimum $x^* = (0, 0)$ faster?

Solution: One Step of Newton's Method

Answers

$$f(x_1, x_2) = x_1^2 + 4x_2^2, x_0 = (4, 2)^T$$

$$(1) \nabla f(x) = (2x_1, 8x_2)^T, \text{ so } \nabla f(x_0) = (8, 16)^T$$

$$\nabla^2 f(x) = \begin{pmatrix} 2 & 0 \\ 0 & 8 \end{pmatrix} \text{ (constant Hessian)}$$

$$(2) [\nabla^2 f(x_0)]^{-1} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/8 \end{pmatrix}$$

$$d_0^N = - \begin{pmatrix} 1/2 & 0 \\ 0 & 1/8 \end{pmatrix} \begin{pmatrix} 8 \\ 16 \end{pmatrix} = - \begin{pmatrix} 4 \\ 2 \end{pmatrix} = \begin{pmatrix} -4 \\ -2 \end{pmatrix}$$

$$(3) x_1 = x_0 + d_0^N = (4, 2)^T + (-4, -2)^T = (0, 0)^T = x^* \checkmark$$

(4) Newton converges in 1 step! Steepest descent (from Exercise 1) needed $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots$

For quadratics, Newton is unbeatable.

Modification 1: Line Search Along Newton Direction

Problem: Newton's method without line search can diverge far from solution.

Solution: Use line search with Newton direction d_k^N :

Newton's Method with Line Search

1. If $\|\nabla f(\mathbf{x}_k)\| < \epsilon$, **stop**. Else compute $\mathbf{d}_k = -[\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k)$.
2. **Line search:** $\lambda_k = \text{backtracking}(\mathbf{x}_k, \mathbf{d}_k)$.
3. Update: $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{d}_k$. Set $k \leftarrow k + 1$, go to Step 1.

Key insight:

- If Hessian is $\succ 0$, then d_k^N is a descent direction
- Line search with Wolfe conditions ensures **global convergence**
- Still maintains **local quadratic convergence**
- $\lambda_k \rightarrow 1$ near solution (Newton step eventually accepted)

Modification 2: Levenberg-Marquardt Method

Problem: Hessian may not be positive definite $\Rightarrow$ Newton direction not descent.

Solution: Add regularization term:

$$M_k = \nabla^2 f(x_k) + \mu_k I_n$$

where $\mu_k \geq 0$ is chosen so that $M_k \succ 0$.

Modified Newton step:

$$d_k^{\text{LM}} = -M_k^{-1} \nabla f(x_k) = -[\nabla^2 f(x_k) + \mu_k I_n]^{-1} \nabla f(x_k)$$

How to choose μ_k :

- Compute eigenvalues of $\nabla^2 f(x_k)$: $\lambda_i(k)$
- Find $\lambda_{\min}(k) = \min_i \lambda_i(k)$
- Set $\mu_k > -\lambda_{\min}(k)$ to ensure $M_k \succ 0$
- Common choice: $\mu_k = -\lambda_{\min}(k) + \delta$ for small $\delta > 0$

Interpolation:

- $\mu_k = 0$: Pure Newton (requires PD Hessian)
- μ_k large: Close to steepest descent ($M_k \approx \mu_k I$)
- Adaptive μ_k : Adjust based on progress

LM: Descent Direction Proof

Claim: If $M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I \succ 0$ and $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$, then $\mathbf{d}_k^{\text{LM}} = -M_k^{-1} \nabla f(\mathbf{x}_k)$ is a **descent direction**.

LM: Descent Direction Proof

Claim: If $M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I \succ 0$ and $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$, then $\mathbf{d}_k^{\text{LM}} = -M_k^{-1} \nabla f(\mathbf{x}_k)$ is a **descent direction**.

Step 1: Compute the directional derivative: Define

$$\theta_k(\lambda) := f(\mathbf{x}_k - \lambda M_k^{-1} \nabla f(\mathbf{x}_k)).$$

LM: Descent Direction Proof

Claim: If $M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I \succ 0$ and $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$, then $\mathbf{d}_k^{\text{LM}} = -M_k^{-1} \nabla f(\mathbf{x}_k)$ is a **descent direction**.

Step 1: Compute the directional derivative: Define

$$\theta_k(\lambda) := f(\mathbf{x}_k - \lambda M_k^{-1} \nabla f(\mathbf{x}_k)).$$

$$\theta'_k(0) = -\nabla f(\mathbf{x}_k)^T M_k^{-1} \nabla f(\mathbf{x}_k)$$

LM: Descent Direction Proof

Claim: If $M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I \succ 0$ and $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$, then

$\mathbf{d}_k^{\text{LM}} = -M_k^{-1} \nabla f(\mathbf{x}_k)$ is a **descent direction**.

Step 1: Compute the directional derivative: Define

$$\theta_k(\lambda) := f(\mathbf{x}_k - \lambda M_k^{-1} \nabla f(\mathbf{x}_k)).$$

$$\theta'_k(0) = -\nabla f(\mathbf{x}_k)^T M_k^{-1} \nabla f(\mathbf{x}_k)$$

Step 2: Apply positive definiteness: Since $M_k \succ 0$, we have

$M_k^{-1} \succ 0$, so:

$$\nabla f(\mathbf{x}_k)^T M_k^{-1} \nabla f(\mathbf{x}_k) > 0 \quad \text{for any } \nabla f(\mathbf{x}_k) \neq \mathbf{0}$$

LM: Descent Direction Proof

Claim: If $M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I \succ 0$ and $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$, then $\mathbf{d}_k^{\text{LM}} = -M_k^{-1} \nabla f(\mathbf{x}_k)$ is a **descent direction**.

Step 1: Compute the directional derivative: Define

$$\theta_k(\lambda) := f(\mathbf{x}_k - \lambda M_k^{-1} \nabla f(\mathbf{x}_k)).$$

$$\theta'_k(0) = -\nabla f(\mathbf{x}_k)^T M_k^{-1} \nabla f(\mathbf{x}_k)$$

Step 2: Apply positive definiteness: Since $M_k \succ 0$, we have $M_k^{-1} \succ 0$, so:

$$\nabla f(\mathbf{x}_k)^T M_k^{-1} \nabla f(\mathbf{x}_k) > 0 \quad \text{for any } \nabla f(\mathbf{x}_k) \neq \mathbf{0}$$

Step 3: Conclude: $\theta'_k(0) < 0$, so $\mathbf{d}_k^{\text{LM}}$ is a descent direction. □

LM: Descent Direction Proof

Claim: If $M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I \succ 0$ and $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$, then

$\mathbf{d}_k^{\text{LM}} = -M_k^{-1} \nabla f(\mathbf{x}_k)$ is a **descent direction**.

Step 1: Compute the directional derivative: Define

$$\theta_k(\lambda) := f(\mathbf{x}_k - \lambda M_k^{-1} \nabla f(\mathbf{x}_k)).$$

$$\theta'_k(0) = -\nabla f(\mathbf{x}_k)^T M_k^{-1} \nabla f(\mathbf{x}_k)$$

Step 2: Apply positive definiteness: Since $M_k \succ 0$, we have

$M_k^{-1} \succ 0$, so:

$$\nabla f(\mathbf{x}_k)^T M_k^{-1} \nabla f(\mathbf{x}_k) > 0 \quad \text{for any } \nabla f(\mathbf{x}_k) \neq \mathbf{0}$$

Step 3: Conclude: $\theta'_k(0) < 0$, so $\mathbf{d}_k^{\text{LM}}$ is a descent direction. □

LM with line search:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \lambda_k M_k^{-1} \nabla f(\mathbf{x}_k), \quad \lambda_k \in \arg \min_{\lambda \geq 0} f(\mathbf{x}_k - \lambda M_k^{-1} \nabla f(\mathbf{x}_k))$$

Under suitable assumptions, Newton's method with the Levenberg-Marquardt modification enjoys **global convergence** (convergence from any starting point).

LM: Interpolation Between SD and Newton

$$M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I_n$$

The LM method smoothly interpolates between steepest descent and Newton's method:

LM: Interpolation Between SD and Newton

$$M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I_n$$

The LM method smoothly interpolates between steepest descent and Newton's method:

- If $\nabla^2 f(\mathbf{x}_k) \succ 0$, we can set $\mu_k = 0$ and LM coincides with pure Newton's method.

LM: Interpolation Between SD and Newton

$$M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I_n$$

The LM method smoothly interpolates between steepest descent and Newton's method:

- If $\nabla^2 f(\mathbf{x}_k) \succ 0$, we can set $\mu_k = 0$ and LM coincides with pure Newton's method.
- On the other hand, if μ_k is set to a large positive value, then $M_k \approx \mu_k I_n$, so $M_k^{-1} \approx \mu_k^{-1} I_n$ and the update becomes:

$$\mathbf{x}_{k+1} \approx \mathbf{x}_k - \lambda_k \mu_k^{-1} \nabla f(\mathbf{x}_k)$$

which is an approximation of steepest descent.

LM: Interpolation Between SD and Newton

$$M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I_n$$

The LM method smoothly **interpolates between steepest descent and Newton's method**:

- If $\nabla^2 f(\mathbf{x}_k) \succ 0$, we can set $\mu_k = 0$ and LM coincides with **pure Newton's method**.
- On the other hand, if μ_k is set to a large positive value, then $M_k \approx \mu_k I_n$, so $M_k^{-1} \approx \mu_k^{-1} I_n$ and the update becomes:

$$\mathbf{x}_{k+1} \approx \mathbf{x}_k - \lambda_k \mu_k^{-1} \nabla f(\mathbf{x}_k)$$

which is an approximation of **steepest descent**.

- In practice, μ_k is **adapted** at each iteration:
 - Start with a moderate μ_k
 - **Decrease** μ_k when progress is good (shift toward Newton)
 - **Increase** μ_k when progress is poor (shift toward SD)

LM: Interpolation Between SD and Newton

$$M_k = \nabla^2 f(\mathbf{x}_k) + \mu_k I_n$$

The LM method smoothly **interpolates between steepest descent and Newton's method**:

- If $\nabla^2 f(\mathbf{x}_k) \succ 0$, we can set $\mu_k = 0$ and LM coincides with **pure Newton's method**.
- On the other hand, if μ_k is set to a large positive value, then $M_k \approx \mu_k I_n$, so $M_k^{-1} \approx \mu_k^{-1} I_n$ and the update becomes:

$$\mathbf{x}_{k+1} \approx \mathbf{x}_k - \lambda_k \mu_k^{-1} \nabla f(\mathbf{x}_k)$$

which is an approximation of **steepest descent**.

- In practice, μ_k is **adapted** at each iteration:
 - Start with a moderate μ_k
 - **Decrease** μ_k when progress is good (shift toward Newton)
 - **Increase** μ_k when progress is poor (shift toward SD)

Key advantage: LM inherits **global convergence** from steepest descent (via large μ_k) and **fast local convergence** from Newton (via small μ_k).

Modification 3: Quasi-Newton Methods

Problem: Computing and storing the Hessian is expensive ($O(n^3)$ or $O(n^2)$ space).

Idea: Approximate the Hessian using only gradient information!

Core principle - Secant Condition:

If B_k is an approximation to $\nabla^2 f(x_k)$, it should satisfy:

$$B_{k+1}(x_{k+1} - x_k) = \nabla f(x_{k+1}) - \nabla f(x_k)$$

or equivalently:

$$B_{k+1}s_k = y_k$$

where:

$$s_k = x_{k+1} - x_k$$

$$y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$$

This says: The approximate Hessian should match the actual change in gradients.

Quasi-Newton Methods: Popular Formulas

SR1 (Symmetric Rank-One) Update:

$$B_{k+1} = B_k + \frac{(y_k - B_k s_k)(y_k - B_k s_k)^T}{(y_k - B_k s_k)^T s_k}$$

- Advantage: **Simple** rank-1 update
- Disadvantage: **May NOT preserve PD** (can lose positive definiteness)

BFGS (Broyden-Fletcher-Goldfarb-Shanno) Update:

$$B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}$$

- Advantage: **Preserves PD** if initial B_0 is PD and $y_k^T s_k > 0$
- Advantage: **Superlinear convergence** (almost as fast as Newton)
- Disadvantage: Rank-2 update, slightly more complex

Computational cost:

- Both: $O(n^2)$ per iteration (vs. $O(n^3)$ for Newton)
- Storage: $O(n^2)$ for Hessian approximation B_k
- For **large** n , can use limited-memory variants (L-BFGS)

Practical Quasi-Newton: Updating the Inverse Directly

Key practical insight: In quasi-Newton methods, we need to solve $B_{k+1}\mathbf{d}_{k+1} = -\nabla f(\mathbf{x}_{k+1})$ at each step. Rather than forming B_{k+1} and then inverting, we can **update B_{k+1}^{-1} directly**.

Practical Quasi-Newton: Updating the Inverse Directly

Key practical insight: In quasi-Newton methods, we need to solve $B_{k+1}\mathbf{d}_{k+1} = -\nabla f(\mathbf{x}_{k+1})$ at each step. Rather than forming B_{k+1} and then inverting, we can **update B_{k+1}^{-1} directly**.

BFGS inverse update: Let $H_k := B_k^{-1}$. Then:

$$H_{k+1} = \left(I - \frac{\mathbf{s}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \right) H_k \left(I - \frac{\mathbf{y}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \right) + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}$$

Practical Quasi-Newton: Updating the Inverse Directly

Key practical insight: In quasi-Newton methods, we need to solve $B_{k+1}\mathbf{d}_{k+1} = -\nabla f(\mathbf{x}_{k+1})$ at each step. Rather than forming B_{k+1} and then inverting, we can **update B_{k+1}^{-1} directly**.

BFGS inverse update: Let $H_k := B_k^{-1}$. Then:

$$H_{k+1} = \left(I - \frac{\mathbf{s}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \right) H_k \left(I - \frac{\mathbf{y}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \right) + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}$$

Advantages:

- The search direction is computed as $\mathbf{d}_{k+1} = -H_{k+1} \nabla f(\mathbf{x}_{k+1})$
(matrix-vector product)
- No linear system solve needed! Cost: $O(n^2)$ per iteration
- Compare: Newton requires $O(n^3)$ per iteration (LU factorization)

Practical Quasi-Newton: Updating the Inverse Directly

Key practical insight: In quasi-Newton methods, we need to solve $B_{k+1}\mathbf{d}_{k+1} = -\nabla f(\mathbf{x}_{k+1})$ at each step. Rather than forming B_{k+1} and then inverting, we can **update B_{k+1}^{-1} directly**.

BFGS inverse update: Let $H_k := B_k^{-1}$. Then:

$$H_{k+1} = \left(I - \frac{\mathbf{s}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \right) H_k \left(I - \frac{\mathbf{y}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \right) + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}$$

Advantages:

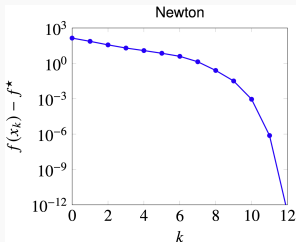
- The search direction is computed as $\mathbf{d}_{k+1} = -H_{k+1} \nabla f(\mathbf{x}_{k+1})$ (**matrix-vector product**)
- No linear system solve needed! Cost: $O(n^2)$ per iteration
- Compare: Newton requires $O(n^3)$ per iteration (LU factorization)

Limited-Memory BFGS (L-BFGS): For **very large** n , even $O(n^2)$ storage for H_k is too much. L-BFGS stores only the last m pairs $\{(\mathbf{s}_i, \mathbf{y}_i)\}$ (typically $m = 3-20$) and implicitly computes $H_k \nabla f(\mathbf{x}_k)$ in $O(mn)$ time.

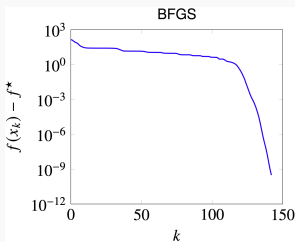
See Section 7.2 of Nocedal and Wright for details.

Numerical Comparison: Newton vs. BFGS

$$n = 100, m = 500: \min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} - \sum_{i=1}^m \ln(b_i - \mathbf{a}_i^T \mathbf{x})$$



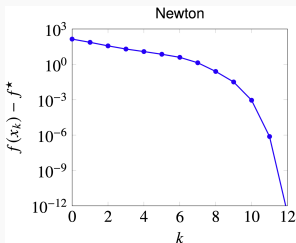
Newton's method
Cost/iteration: $O(n^3)$



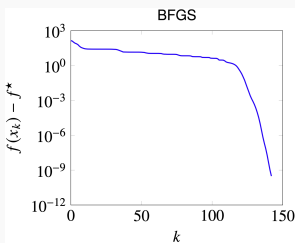
BFGS method
Cost/iteration: $O(n^2)$

Numerical Comparison: Newton vs. BFGS

$$n = 100, m = 500: \min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{c}^T \mathbf{x} - \sum_{i=1}^m \ln(b_i - \mathbf{a}_i^T \mathbf{x})$$



Newton's method
Cost/iteration: $O(n^3)$



BFGS method
Cost/iteration: $O(n^2)$

Observations:

- Newton converges in fewer iterations (quadratic rate)
- BFGS takes more iterations but each is **much cheaper**
- In total wall-clock time, BFGS often wins for large n

Quasi-Newton Algorithm (BFGS)

Algorithm: BFGS Method

Initialization: Choose $\mathbf{x}_1 \in \mathbb{R}^n$, set $B_1 = I_n$, $k = 1$.

1. If $\|\nabla f(\mathbf{x}_k)\| < \epsilon$, **stop**. Else compute $\mathbf{d}_k = -B_k^{-1}\nabla f(\mathbf{x}_k)$.
2. Line search: $\lambda_k = \text{backtracking}(\mathbf{x}_k, \mathbf{d}_k)$.
3. Update: $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda_k \mathbf{d}_k$.
4. Compute: $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$, $\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$.
5. **Update Hessian approximation** B_{k+1} via BFGS formula. Set $k \leftarrow k + 1$, go to Step 1.

Practical advantages:

- No Hessian computation needed (only gradients)
- **Superlinear convergence** (much faster than steepest descent)
- Cheaper per iteration than Newton (no linear system solve)
- **Robust**: Wolfe line search ensures global convergence

Exercise 4: Secant Condition

Secant Condition

Suppose we have $x_k = (1, 1)^T$, $x_{k+1} = (1.1, 0.95)^T$.

At these points: $\nabla f(x_k) = (2, 1)^T$ and $\nabla f(x_{k+1}) = (2.2, 0.9)^T$.

Suppose our Hessian approximation is $B_k = I$ (identity matrix).

1. Compute $s_k = x_{k+1} - x_k$ and $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$
2. Check if the secant condition $B_k s_k = y_k$ is satisfied
3. Compute the residual: $B_k s_k - y_k$

Solution: Secant Condition

Answers

(1)

$$s_k = x_{k+1} - x_k = (1.1, 0.95)^T - (1, 1)^T = (0.1, -0.05)^T$$

$$y_k = \nabla f(x_{k+1}) - \nabla f(x_k) = (2.2, 0.9)^T - (2, 1)^T = (0.2, -0.1)^T$$

(2) Check if $B_k s_k = y_k$:

$$B_k s_k = I \cdot (0.1, -0.05)^T = (0.1, -0.05)^T$$

But $y_k = (0.2, -0.1)^T$, so:

$$B_k s_k \neq y_k$$

The secant condition is **NOT satisfied**.

(3) Residual:

$$B_k s_k - y_k = (0.1, -0.05)^T - (0.2, -0.1)^T = (-0.1, 0.05)^T$$

This means $B_k = I$ is a poor approximation to the true Hessian. The BFGS update would adjust B_k to satisfy the secant condition better.

Exercise 5: Levenberg-Marquardt Modification

Levenberg-Marquardt Modification

Consider $f(x_1, x_2) = x_1^4 + 2x_2^2 - 2x_1x_2$ at $x_0 = (1, 1)^T$.

1. Compute $\nabla f(x_0)$ and $\nabla^2 f(x_0)$
2. Find the eigenvalues of $\nabla^2 f(x_0)$
3. Is the Hessian positive definite?
4. If not, what value of μ_0 would make $M_0 = \nabla^2 f(x_0) + \mu_0 I$ positive definite?

Solution: Levenberg-Marquardt Modification

Answers

$$f(x_1, x_2) = x_1^4 + 2x_2^2 - 2x_1x_2, \quad x_0 = (1, 1)^T$$

$$(1) \quad \nabla f = (4x_1^3 - 2x_2, 4x_2 - 2x_1)^T$$

$$\text{At } x_0: \nabla f(x_0) = (4 - 2, 4 - 2)^T = (2, 2)^T$$

$$\nabla^2 f = \begin{pmatrix} 12x_1^2 & -2 \\ -2 & 4 \end{pmatrix}$$

$$\text{At } x_0: \nabla^2 f(x_0) = \begin{pmatrix} 12 & -2 \\ -2 & 4 \end{pmatrix}$$

(2) Eigenvalues:

$$\det(\lambda I - H) = (\lambda - 12)(\lambda - 4) - 4 = \lambda^2 - 16\lambda + 44 = 0$$

$$\lambda = \frac{16 \pm \sqrt{256 - 176}}{2} = \frac{16 \pm \sqrt{80}}{2} = 8 \pm 2\sqrt{5}$$

$$\lambda_1 \approx 12.47, \quad \lambda_2 \approx 3.53$$

(3) Both eigenvalues are **positive**, so Hessian **IS PD!** ✓

(4) Since $\lambda_{\min} > 0$, we can use Newton's method with $\mu_0 = 0$. If needed, even $\mu_0 = 1$ would keep M_0 PD.

Exercise 6: Method Comparison

Exercise 6: Method Comparison

Match each method to its primary advantage:

1. Steepest Descent
2. Newton's Method (with line search)
3. Levenberg-Marquardt
4. BFGS

Advantages:

- (A) No Hessian computation; superlinear convergence
- (B) Works even when Hessian is not PD
- (C) Simplicity; guaranteed descent; global convergence
- (D) Quadratic convergence; handles ill-conditioned problems

Solution: Method Comparison

Solution 6

Correct pairings:

- (1) → (C) **Steepest Descent**: Simple, guaranteed descent, global convergence.
- (2) → (D) **Newton's Method**: Quadratic convergence; handles ill-conditioned problems via Hessian.
- (3) → (B) **Levenberg-Marquardt**: Adds $\mu_k I$ to ensure PD when Hessian is not.
- (4) → (A) **BFGS**: Avoids Hessian computation (only needs gradients); achieves superlinear convergence.

Summary: For practitioners:

- Hessian cheap? → Newton + line search
- Hessian expensive or indefinite? → BFGS or Levenberg-Marquardt
- Large n ($n > 10^4$) and gradient OK? → L-BFGS

Key Takeaways

Steepest Descent Method

- Simplest gradient-based method; guaranteed convergence
- Linear convergence rate; slow on ill-conditioned problems
- Per-iteration cost: $O(n)$ + line search

Inexact Line Search (Wolfe Conditions)

- Armijo (sufficient decrease) + Curvature condition
- Backtracking is simple and efficient
- Practical trade-off: good progress without exact minimization

Newton's Method

- Uses Hessian for curvature; quadratic convergence near solution
- One step for quadratics; very fast when Hessian available
- Per-iteration cost: $O(n^3)$; requires line search for global convergence

Modifications

- **Levenberg-Marquardt**: Ensures PD by adding $\mu_k I$
- **BFGS**: Avoids Hessian; superlinear convergence; **most popular** in practice
- **L-BFGS**: Limited-memory variant for large-scale problems

Next Lecture (Lecture 12): Conjugate Direction Methods

- **Conjugate directions**: A powerful concept for quadratics
- **Conjugate gradient method**: Combines efficiency of Newton with simplicity of SD
- Convergence in at most n steps (no rounding!)
- Extends to nonlinear problems
- Practical iterative method for large-scale optimization

Why care?

- Bridges gap between simple (SD) and powerful (Newton) methods
- Foundation for modern optimization solvers
- Essential for constrained optimization (next unit)

Questions?

ISE 5406 — Optimization II
Virginia Tech, Spring 2026